Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

LAB 09	
Aim:	Design an event driven solution for the given problem.
Theory:	In an event-driven architecture for the Cab Booking System, the core idea is that the system doesn't follow a fixed sequence of operations; instead, it remains idle until an event occurs. These events can be triggered by user interactions—such as clicking a "Book Cab" button, submitting a form to cancel a ride, or refreshing the list of available cabs—or by system-generated actions, like updates to cab availability. Each event is encapsulated in an object that carries relevant data (e.g., passenger name, cab ID), and a corresponding event handler processes this data to perform the required operation. This model enables a clear separation of concerns. The user interface becomes responsible only for collecting inputs and displaying results, while the business logic remains in dedicated handler functions. For instance, when a BookRideEvent is triggered, the system's dispatcher routes it to a handleBookRide() method, which manages cab availability and booking records. Similarly, cancellation events are handled by specific methods that update the state of both the booking list and the cab's available seats. Event-driven design improves the scalability of the application, as it is easier to add new types of events without disturbing the existing code structure. It also enhances reusability, since components like handlers and event classes can be reused across different platforms, such as web apps, mobile apps, or desktop applications. Moreover, in real-time applications where responsiveness is critical, such as live cab tracking or instant booking confirmation, the event-driven approach provides faster and more dynamic updates. Overall, transitioning to an event-driven solution modernizes the Cab Booking System, making it more interactive, maintainable, and suitable for integration with various user interfaces and platforms. This design lays the foundation for building a real-world application that can easily evolve with changing user needs and technological advancements.

Program:	//Sheikh Wasimuddin 23071044 CSEIOT-73
	import java.util.*;
	
	class Cab {
	  private final int cabId;
	  private final String cabModel;
	  private final String driverName;
	  private int availableSeats;
	
	  public Cab(int cabId, String cabModel, String driverName, int availableSeats) {
	  this.cabId = cabId;
	  this.cabModel = cabModel;

```
        this.driverName = driverName;
        this.availableSeats = availableSeats;
    }

    public int getCabId() {
        return cabId;
    }

    public String getCabModel() {
        return cabModel;
    }

    public String getDriverName() {
        return driverName;
    }

    public int getAvailableSeats() {
        return availableSeats;
    }

    public boolean bookCab() {
        if (availableSeats > 0) {
            availableSeats--;
            return true;
        }
        return false;
    }

    public void cancelRide() {
        availableSeats++;
    }

    @Override
    public String toString() {
        return "Cab ID: " + cabId + ", Model: " + cabModel + ", Driver: " + driverName
+ ", Available Seats: " + availableSeats;
    }
}

class RideBooking {
    private static int bookingCounter = 1000;
    private final int bookingId;
    private final String passengerName;
    private final Cab cab;

    public RideBooking(String passengerName, Cab cab) {
        this.passengerName = passengerName;
        this.cab = cab;
        this.bookingId = ++bookingCounter;
```

Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

```
    }

    public int getBookingId() {
        return bookingId;
    }

    public String getPassengerName() {
        return passengerName;
    }

    public Cab getCab() {
        return cab;
    }

    @Override
    public String toString() {
        return "Booking ID: " + bookingId + ", Passenger: " + passengerName + ", Cab:
" + cab.getCabModel() + ", Driver: " + cab.getDriverName();
    }
}

public class CabBookingSystem {

    private final List<Cab> cabs = new ArrayList<>();
    private final List<RideBooking> rideBookings = new ArrayList<>();

    public CabBookingSystem() {
        // Predefined cabs
        cabs.add(new Cab(1, "Toyota Prius", "John Doe", 4));
        cabs.add(new Cab(2, "Honda Civic", "Jane Smith", 3));
        cabs.add(new Cab(3, "Ford Mustang", "Jake Peralta", 2));
    }

    public void showAvailableCabs() {
        System.out.println("\n--- Available Cabs ---");
        for (Cab cab : cabs) {
            System.out.println(cab);
        }
    }

    public Cab findCabById(int cabId) {
        for (Cab cab : cabs) {
            if (cab.getCabId() == cabId) {
                return cab;
            }
        }
        return null;
    }
```

Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

```
public void bookRide(String name, int cabId) {
    Cab cab = findCabById(cabId);
    if (cab != null) {
        if (cab.bookCab()) {
            RideBooking booking = new RideBooking(name, cab);
            rideBookings.add(booking);
            System.out.println("Booking successful!\n" + booking);
        } else {
            System.out.println("No seats available.");
        }
    } else {
        System.out.println("Cab not found.");
    }
}

public void cancelRide(int bookingId) {
    Iterator<RideBooking> iterator = rideBookings.iterator();
    while (iterator.hasNext()) {
        RideBooking booking = iterator.next();
        if (booking.getBookingId() == bookingId) {
            booking.getCab().cancelRide();
            iterator.remove();
            System.out.println("Ride cancelled successfully.");
            return;
        }
    }
    System.out.println("Booking ID not found.");
}

public void viewAllBookings() {
    if (rideBookings.isEmpty()) {
        System.out.println("No bookings available.");
    } else {
        System.out.println("\n--- All Bookings ---");
        for (RideBooking booking : rideBookings) {
            System.out.println(booking);
        }
    }
}

public static void main(String[] args) {
    CabBookingSystem system = new CabBookingSystem();
    Scanner sc = new Scanner(System.in);
    int choice;

    do {
        System.out.println("\n===== Cab Booking Menu =====");
        System.out.println("1. View Available Cabs");
        System.out.println("2. Book a Ride");
```

Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

```
            System.out.println("3. Cancel a Ride");
            System.out.println("4. View All Bookings");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1 -> system.showAvailableCabs();
                case 2 -> {
                    sc.nextLine(); // consume leftover newline
                    System.out.print("Enter passenger name: ");
                    String name = sc.nextLine();
                    System.out.print("Enter cab ID: ");
                    int cabId = sc.nextInt();
                    system.bookRide(name, cabId);
                }
                case 3 -> {
                    System.out.print("Enter Booking ID to cancel: ");
                    int bookingId = sc.nextInt();
                    system.cancelRide(bookingId);
                }
                case 4 -> system.viewAllBookings();
                case 5 -> System.out.println("Thank you for using the Cab Booking
System!");
                default -> System.out.println("Invalid choice. Try again.");
            }

        } while (choice != 5);

        sc.close();
    }
}
```

Output:

Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

```
===== Cab Booking Menu =====
1. View Available Cabs
2. Book a Ride
3. Cancel a Ride
4. View All Bookings
5. Exit
Enter your choice: 1

--- Available Cabs ---
Cab ID: 1, Model: Toyota Prius, Driver: John Doe, Available Seats: 4
Cab ID: 2, Model: Honda Civic, Driver: Jane Smith, Available Seats: 3
Cab ID: 3, Model: Ford Mustang, Driver: Jake Peralta, Available Seats: 2

===== Cab Booking Menu =====
1. View Available Cabs
2. Book a Ride
3. Cancel a Ride
4. View All Bookings
5. Exit
Enter your choice: 2
Enter passenger name: Sheikh Wasimuddin
Enter cab ID: 1
Booking successful!
Booking ID: 1001, Passenger: Sheikh Wasimuddin, Cab: Toyota Prius, Driver: John Doe
```

Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

```
===== Cab Booking Menu =====
1. View Available Cabs
2. Book a Ride
3. Cancel a Ride
4. View All Bookings
5. Exit
Enter your choice: 2
Enter passenger name: Sheikh wasim
Enter cab ID: 1
Booking successful!
Booking ID: 1002, Passenger: Sheikh wasim, Cab: Toyota Prius, Driver: John Doe

===== Cab Booking Menu =====
1. View Available Cabs
2. Book a Ride
3. Cancel a Ride
4. View All Bookings
5. Exit
Enter your choice: 4

--- All Bookings ---
Booking ID: 1001, Passenger: Sheikh Wasimuddin, Cab: Toyota Prius, Driver: John Doe
Booking ID: 1002, Passenger: Sheikh wasim, Cab: Toyota Prius, Driver: John Doe
```

Name: SHEIKH WASIMUDDIN
Roll No: IOT-73
Branch: CSE(IOT)

	``` ===== Cab Booking Menu ===== 1. View Available Cabs 2. Book a Ride 3. Cancel a Ride 4. View All Bookings 5. Exit Enter your choice: 3 Enter Booking ID to cancel: 1001 Ride cancelled successfully. ===== Cab Booking Menu ===== 1. View Available Cabs 2. Book a Ride 3. Cancel a Ride 4. View All Bookings 5. Exit Enter your choice: 4 --- All Bookings --- Booking ID: 1002, Passenger: Sheikh wasim, Cab: Toyota Prius, Driver: John Doe ===== Cab Booking Menu ===== 1. View Available Cabs 2. Book a Ride 3. Cancel a Ride 4. View All Bookings 5. Exit Enter your choice: 5 Thank you for using the Cab Booking System! ```
Conclusion	Event driven Program in java is executed successfully.
Name Of Faculty:	Prof. Akhil Jajulwar